

Assignment03: CSC323 Compiler Construction, Spring 2026

Due Date: May 23, 2026, 10:00 PM (sharp)

Submission: Single .zip File upload via LMS

AN HONEST ZERO IS BETTER THAN A STOLEN ONE HUNDRED

Instructions: Welcome to the second project milestone for the compiler construction class! In your first milestone you have almost achieved the front end of the compiler, which includes the scanner and top-down parser. Your front end reads the source file in CL language specified in the previous milestone, identifies the tokens, and checks the syntax of your program. Now it's time to work on the Semantic analyzer and part of the back end of the compiler that is Intermediate code generation.

You must organize your project as follows:

1. Front-end
 - a. .jjt file that includes your scanner and parser definition.
 - b. A java file that contains data structure for symbol table.
 - c. A java file that contains functionality of semantic analyser.
2. Back-end
 - a. A java file that performs intermediate code generation.
3. Documentation (**Latex file only**)
4. You must zip all your file and the name of your zipped file should be as follows:
S26_6D_001_002__003

All the files using other format will not be considered. Any late submissions will not be considered.

Academic Integrity: Collaboration is permitted within project limits (max three students). All submitted work must be original. **Plagiarism** is strictly prohibited and will be **penalized according to university policy**.

Additions for the Project:

Implement the following:

1. Data types must include int, float string and char.
2. Symbol Table must store exact lexeme of the variable, its type and value.
3. Semantic analyser must use the Symbol table to find semantic errors.
4. You must have a separate class for semantic analysis.
5. Conditional and loop statements discussed in the project I must also be implemented.

Semantic analyser:

Your semantic analyser must explore the AST tree for the assignment statements and expressions to look for any type error. For example, if in a program you have b defined as a string and c defined as an integer, then the following assignment statement contains a type error and your compiler must stop and produce error stating the type of error and line number where it found it.

$a = b + c$

All the operations must always be applied to variables of same type.

Intermediate representation description:

In intermediate code generation phase, you need to produce three-address code for the syntax tree of the source code. Each instruction in the tree must be represented using three-address instructions. You can follow the quadruple representation for this purpose. For example, if we have an assignment statement $a = b * -c + b * -c$; its intermediate representation must be as follows:

$t_1 = \text{minus } c$

$t_2 = b * t_1$

$t_3 = \text{minus } c$

$t_4 = b * t_3$

$t_5 = t_2 + t_4$

$a = t_5$

	OP	ARG1	ARG2	RESULT
0	<i>minus</i>	<i>c</i>		<i>t₁</i>
1	*	<i>b</i>	<i>t₁</i>	<i>t₂</i>
2	<i>minus</i>	<i>c</i>		<i>t₃</i>
3	*	<i>b</i>	<i>t₃</i>	<i>t₄</i>
4	+	<i>t₂</i>	<i>t₄</i>	<i>t₅</i>
5	=	<i>t₅</i>		<i>a</i>

You must use an array to represent the instructions in the three-address code. Details for the other statements including conditional and loop statements can be found in the book. You must show the contents of the array on the screen.

Good Luck!